Exercise 2 — Concepts

Optimization-Based Control Methods · Lecture 2: Dynamic Programming (Finite Horizon)

Course: Optimization-Based Control Methods (TU Chemnitz, Automatic Control Group, Dr. Adrian Redder) **Lecture:** 2 — Dynamic Programming (script: Chapter 2) **Covers:** everything needed for Exercise 2 (Tasks 1–4, the numerical experiment and the additional task)

This document builds dynamic programming (DP) from the ground up. It assumes only what was covered in Exercise 1. Every formula is derived, and no steps are skipped. The worked solutions are in `02_Solutions.md`, and the Python scripts in `python/` check every number used here.

Notation follows the lecture: x_k state, u_k control, w_k disturbance, f_k dynamics, g_k stage cost, g_N terminal cost, μ_k feedback rule, $\pi = \{\mu_0, \dots, \mu_{N-1}\}$ policy, J_k cost-to-go. A star ($J^\star$, $\mu^\star$, $u^\star$) means "optimal".

Contents

1. Mathematical toolkit
2. From static to sequential optimisation
3. The general stochastic DP problem
4. Policies and the optimal cost
5. Bellman's principle of optimality
6. The DP algorithm
7. Worked lecture example: thermal pre-heating
8. Further lecture examples
9. Designing cost functions
10. Linear-quadratic DP (scalar)
11. Discounted cost
12. Tabular DP with hard constraints
13. Monotonicity of DP
14. Outlook: infinite horizon
15. Cheat sheet

---

1. Mathematical toolkit

DP is built from three ingredients: minimising a function, taking expectations, and induction. This section collects the facts used later.

1.1 min, argmin and ties

For a function $h(u)$ over a set U:

- $\min_{u \in U} h(u)$ is the **smallest value** of h (a number).

- $\arg\min_{u \in U} h(u)$ is the **set of inputs** that achieve this value.

Example: $h(u) = (u-2)^2 + 1$ on $U = \mathbb{R}$ gives $\min h = 1$ and $\arg\min h = \{2\}$.

If several inputs give the same smallest value, the argmin has several elements. This is called a **tie**, and any element of the argmin is optimal. Ties appear in Task 4. When U is finite (for example $U = \{0, 1\}$), the minimum is found by evaluating h at every element and picking the smallest value. No derivatives are needed.

Two rules used throughout:

- **Constants come out of a min:** $\min_u \big(h(u) + c\big) = c + \min_u h(u)$ when c does not depend on u.
- **Maximising is minimising the negative:** $\max_u h(u) = -\min_u \big(-h(u)\big)$, with the same argmax/argmin. This is why the course treats only minimisation.

1.2 Minimising a smooth function of one variable

For a differentiable $h : \mathbb{R} \to \mathbb{R}$:

1. **Necessary condition.** At an interior minimiser $u^\star$ the slope is zero: $h'(u^\star) = 0$. Reason: if $h'(u^\star) > 0$, moving slightly left would decrease h; if $h'(u^\star) < 0$, moving slightly right would.
2. **Sufficient condition.** If in addition $h''(u) > 0$ for all u, the function is **strictly convex** (it curves upwards everywhere), so the stationary point is the unique global minimiser.

For a quadratic $h(u) = \alpha u^2 + 2\beta u + \gamma$ the second derivative is the constant $h'' = 2\alpha$, so $\alpha > 0$ is all that is needed.

1.3 Completing the square and the key lemma

General quadratic. Let $\alpha > 0$. Then

$$\alpha u^2 + 2\beta u + \gamma = \alpha\Big(u^2 + \tfrac{2\beta}{\alpha} u\Big) + \gamma = \alpha\Big(u + \tfrac{\beta}{\alpha}\Big)^2 - \tfrac{\beta^2}{\alpha} + \gamma.$$

(Check: expanding $\alpha(u + \beta/\alpha)^2 = \alpha u^2 + 2\beta u + \beta^2/\alpha$, so subtracting β^2/α gives back the left side.) The square is never negative and is zero only for $u = -\beta/\alpha$. Hence

$$\boxed{\min_u \ (\alpha u^2 + 2\beta u + \gamma) = \gamma - \frac{\beta^2}{\alpha}, \qquad u^\star = -\frac{\beta}{\alpha}}$$

Key lemma (used in almost every DP derivation of this exercise). Let $r > 0$, $p \geq 0$ and let c, β be numbers that do not depend on u. Then

$$\boxed{\min_{u \in \mathbb{R}} \Big[r u^2 + p\,(c + \beta u)^2 \Big] = \frac{r\,p}{r + \beta^2 p}\, c^2, \qquad u^\star = -\frac{\beta p}{r + \beta^2 p}\, c}$$

Derivation. Call the bracket $\varphi(u)$.

- Derivative: $\varphi'(u) = 2ru + 2p\beta(c + \beta u)$ (chain rule on the square).
- Second derivative: $\varphi''(u) = 2r + 2p\beta^2 = 2(r + \beta^2 p) > 0$, so φ is strictly convex (§1.2).
- Set $\varphi'(u) = 0$: $ru + p\beta c + p\beta^2 u = 0 \ \Rightarrow\ (r + \beta^2 p)u = -p\beta c \ \Rightarrow\ u^\star = -\dfrac{\beta p\, c}{r + \beta^2 p}$.

- Value. First compute the bracket inside the square at $u^\star$:

$$c + \beta u^\star = c - \frac{\beta^2 p\, c}{r + \beta^2 p} = \frac{c\,(r + \beta^2 p) - \beta^2 p\, c}{r + \beta^2 p} = \frac{r\, c}{r + \beta^2 p}.$$

  Then

$$\varphi(u^\star) = r\,\frac{\beta^2 p^2 c^2}{(r + \beta^2 p)^2} + p\,\frac{r^2 c^2}{(r + \beta^2 p)^2} = \frac{r\, p\, c^2\,(\beta^2 p + r)}{(r + \beta^2 p)^2} = \frac{r\, p\, c^2}{r + \beta^2 p}.$$

With $r = 1$ (the case of most examples): $\min_u[u^2 + p(c + \beta u)^2] = \dfrac{p\, c^2}{1 + \beta^2 p}$ at $u^\star = -\dfrac{\beta p\, c}{1 + \beta^2 p}$.

Interpretation. The input pays ru^2 but reduces the "error" c that is penalised with weight p. The optimum does not remove the error completely: it leaves the fraction $r/(r + \beta^2 p)$ of it (computed above). The larger p (error is expensive) or β (input is effective), the more of the error is removed.

1.4 Probability essentials

A **random variable** W takes values at random. For a discrete W with values $w^{(i)}$ and probabilities $P(W = w^{(i)})$:

- **Expectation (mean):** $\mathbb{E}[h(W)] = \sum_i h(w^{(i)})\, P(W = w^{(i)})$. For a continuous W with density ρ: $\mathbb{E}[h(W)] = \int h(w)\rho(w)\, dw$.
- **Linearity:** $\mathbb{E}[\alpha X + \beta Y + \gamma] = \alpha\mathbb{E}[X] + \beta\mathbb{E}[Y] + \gamma$ for numbers α, β, γ. This follows directly from the sum (or integral) being linear.
- **Constants:** $\mathbb{E}[c] = c$. Anything that is fixed when the expectation is taken (for example x_k and u_k when averaging over W_k) behaves like a constant.
- **Zero mean and variance:** if $\mathbb{E}[W] = 0$, then $\mathrm{Var}(W) = \mathbb{E}[(W - 0)^2] = \mathbb{E}[W^2] = \sigma^2$.
- **The expansion used in Task 2(c):** for numbers p, m and $\mathbb{E}[W] = 0$, $\mathbb{E}[W^2] = \sigma^2$:

$$\mathbb{E}[p\,(m + W)^2] = \mathbb{E}[pm^2 + 2pmW + pW^2] = pm^2 + 2pm \cdot 0 + p\sigma^2 = p\, m^2 + p\, \sigma^2.$$

  Noise with zero mean adds a constant and does not change the dependence on m.
- **Independence:** X and Y are independent if knowing X tells nothing about Y: $P(Y = y \mid X = x) = P(Y = y)$. **i.i.d.** means independent and identically distributed (every W_k has the same distribution).
- **Conditional expectation** $\mathbb{E}[Y \mid X = x]$: the average of Y when we already know $X = x$.
- **Law of total expectation:** $\mathbb{E}[Y] = \mathbb{E}_X\big[\mathbb{E}[Y \mid X]\big]$. Derivation for discrete variables:

$$\mathbb{E}[Y] = \sum_y y\, P(Y = y) = \sum_y y \sum_x P(Y = y, X = x) = \sum_x P(X = x) \underbrace{\sum_y y\, P(Y = y \mid X = x)}_{\mathbb{E}[Y|X=x]}.$$

  In words: average first with X fixed, then average over X. This "average in two stages" is exactly what DP does with the disturbances, one stage at a time.

1.5 Mathematical induction (forwards and backwards)

To prove a statement $S(k)$ for all k in a range:

1. **Base case:** prove S for the first index.
2. **Induction step:** prove "if S holds for one index, it holds for the next".

DP runs **backwards** in time, so most proofs here start at $k = N$ (base case) and step from $k + 1$ to k. The logic is identical: $S(N)$ true, $S(N) \Rightarrow S(N - 1)$, $S(N - 1) \Rightarrow S(N - 2)$, and so on down to $S(0)$.

1.6 Geometric series

For $|\alpha| < 1$: $\sum_{k=0}^{N-1} \alpha^k = \dfrac{1-\alpha^N}{1-\alpha}$ and $\sum_{k=0}^{\infty} \alpha^k = \dfrac{1}{1-\alpha}$.

Derivation: let $S = 1 + \alpha + \cdots + \alpha^{N-1}$. Then $\alpha S = \alpha + \cdots + \alpha^N$, and subtracting gives $S - \alpha S = 1 - \alpha^N$, so $S = (1-\alpha^N)/(1-\alpha)$. As $N \to \infty$, $\alpha^N \to 0$.

---

2. From static to sequential optimisation

2.1 Static optimisation

A **static** problem chooses one decision once:

$$\min_{u \in U} g(u) \qquad \text{or, with randomness,} \qquad \min_{u \in U} \mathbb{E}_W\big[G(u, W)\big].$$

Examples from the lecture: choosing one of K cities for a warehouse (discrete U), optimising a wing profile (continuous $U \subseteq \mathbb{R}^m$), least squares (convex), and training a classifier on random samples (stochastic).

2.2 Sequential (dynamic) optimisation

In a **sequential** problem, decisions $u_0, u_1, \ldots$ are taken in **stages** $k = 0, 1, \ldots$ while a state x_k evolves. Three features make it different:

1. **Temporal coupling.** Each u_k changes all future states $x_{k+1}, x_{k+2}, \ldots$, so decisions cannot be optimised one at a time greedily.
2. **Information timing.** At stage k the controller sees x_k, applies u_k, and only then is the disturbance w_k realised. The decision can use everything up to x_k but nothing after.
3. **Feedback is possible.** Since the state is observed again at every stage, later decisions can correct for earlier surprises.

2.3 The running example: thermal pre-heating

A furnace (or an EV battery) must be heated to a target temperature T by a deadline stage N:

$$x_{k+1} = (1-a)\,x_k + a\,u_k\ (+\,w_k), \qquad J(x_0; u_0, \ldots, u_{N-1}) = \underbrace{r\,(x_N - T)^2}_{\text{miss the target}} + \underbrace{\sum_{k=0}^{N-1} u_k^2}_{\text{heating energy}}.$$

- x_k: temperature at stage k; u_k: heater input; w_k: random disturbance (a cold draft).
- $a \in (0,1)$: heat transfer coefficient. With $u_k = 0$ the temperature becomes $(1-a)x_k$: a fraction $(1-a)$ is **retained**, the rest is lost.
- $r > 0$: how much a terminal miss matters compared to energy.

The two questions that make this a sequential problem:

- **Heat early or late?** Heat injected early leaks away (a factor $(1-a)$ per stage). Heat injected at the last stage does not leak, but a single large input costs a lot, because the cost u^2 grows quadratically. The optimum must balance these (§7.4 shows how).
- **Timer or thermostat?** If a cold draft hits mid-process, should the plan be fixed in advance or adapted? This is the next subsection.

2.4 Open-loop sequences vs. closed-loop policies

- **Open-loop control sequence** ("timer"): all inputs $(u_0, \dots, u_{N-1})$ are fixed at $k = 0$, based only on x_0. It cannot react to disturbances.
- **Closed-loop feedback policy** ("thermostat"): a sequence of **rules** $\pi = \{\mu_0, \dots, \mu_{N-1}\}$. At stage k the controller measures x_k and applies $u_k = \mu_k(x_k)$. A rule is a function: it says what to do for **every** state that might occur.

Value of information.

- *Deterministic system (exact model, no w_k):* the state at each stage can be predicted exactly from x_0 and the inputs, so measuring it adds nothing. The best open-loop sequence and the best policy give the same trajectory and the same cost.
- *Stochastic system:* waiting until x_k is measured lets u_k compensate for the disturbances $w_0, \dots, w_{k-1}$ that already happened. A policy can never be worse than an open-loop sequence (an open-loop sequence is a special policy that ignores the measurement), and it is usually strictly better.

2.5 A two-stage example of the value of information

System $x_{k+1} = x_k + u_k + w_k$, $N = 2$, cost $\mathbb{E}[x_2^2]$ (no input cost), w_0, w_1 independent with $\mathbb{E}[w_k] = 0$ and $\mathbb{E}[w_k^2] = \sigma^2$.

- **Open loop.** $x_2 = x_0 + u_0 + u_1 + w_0 + w_1$. With $m := x_0 + u_0 + u_1$ (fixed in advance): $\mathbb{E}[x_2^2] = m^2 + 2m\,\mathbb{E}[w_0 + w_1] + \mathbb{E}[(w_0 + w_1)^2] = m^2 + 0 + \sigma^2 + 0 + \sigma^2$ (the cross term $2\mathbb{E}[w_0 w_1] = 2\mathbb{E}[w_0]\mathbb{E}[w_1] = 0$ by independence). The best choice is $m = 0$, giving $\mathbb{E}[x_2^2] = 2\sigma^2$.
- **Closed loop.** Choose $u_1 = \mu_1(x_1) = -x_1$ after measuring x_1. Then $x_2 = x_1 - x_1 + w_1 = w_1$ and $\mathbb{E}[x_2^2] = \sigma^2$, only **half** the open-loop cost.

The feedback rule cancels w_0 because it acts after w_0 is known. Nothing can cancel w_1, since no decision is taken after it (this is Task 1(b)).

---

3. The general stochastic DP problem

3.1 The system

$$x_{k+1} = f_k(x_k, u_k, w_k), \qquad k = 0, 1, \dots, N-1.$$

- **State** $x_k \in X \subseteq \mathbb{R}^n$: everything about the past that matters for the future.
- **Control** $u_k \in U_k(x_k) \subseteq \mathbb{R}^m$: the set of admissible inputs may depend on the state and the stage (for example: "the battery cannot be charged beyond full").
- **Disturbance** w_k, drawn from a distribution W_k.
- **Time-invariant** means $f_k = f$ for all k.

An equivalent description is a **Markov decision process (MDP)**: instead of a formula with noise, give the probability distribution of the next state, $x_{k+1} \sim P_k(\cdot \mid x_k, u_k)$. Example: $x_{k+1} = x_k + u_k + w_k$ with $w_k \in \{-1, 0, 1\}$ equally likely is the same as "the next state is $x_k + u_k - 1$, $x_k + u_k$ or $x_k + u_k + 1$, each with probability $1/3$".

3.2 The cost

$$\mathbb{E}_{W_0,\dots,W_{N-1}}\Big[\, g_N(x_N) + \sum_{k=0}^{N-1} g_k(x_k, u_k, w_k)\Big].$$

- **Stage cost** g_k: the price paid at stage k. It may be negative, which then means a reward (for example regenerative braking returns energy).
- **Terminal cost** $g_N(x_N)$: the price of where the system ends up at the horizon N.
- **Additive:** the total is a sum of per-stage terms. This is what DP exploits.
- **Expected:** the states are random (they depend on the w_k), so the cost is averaged over all disturbance sequences. With a joint density h this is a big integral over $(w_0, \dots, w_{N-1})$. DP will replace it with N small integrals over one w_k each.

3.3 Standing assumptions

1. **Markovian state feedback:** $u_k = \mu_k(x_k)$ depends only on the current state. The script stresses that this is not really a restriction: if the decision should depend on older information, put that information into the state (**state augmentation**, §9.6).
2. **Conditionally independent noise:** the distribution of w_k may depend on (x_k, u_k) but not on the earlier disturbances $w_{k-1}, \dots, w_0$.

Together they ensure that, once x_k is known, the past does not help predict the future: the state is a complete summary.

3.4 Causality: who can influence what

Unrolling the dynamics shows how the disturbances propagate:

$$x_1 = f_0(x_0, u_0, w_0), \quad x_2 = f_1(x_1, u_1, w_1), \ \dots, \ x_N = f_{N-1}(x_{N-1}, u_{N-1}, w_{N-1}).$$

- w_k appears in x_{k+1}, therefore in every later state $x_{k+1}, \dots, x_N$, every later input $u_{k+1}, \dots$ (through feedback), every later stage cost $g_{k+1}, \dots, g_{N-1}$, the terminal cost g_N, and also in g_k itself if g_k depends on its argument w_k.
- w_k can never influence $x_0, \dots, x_k$ or $u_0, \dots, u_k$: those happen before it.
- Feedback can react to w_k only through $u_{k+1}, \dots, u_{N-1}$. For w_{N-1} no such input exists.

3.5 Cost criteria for long horizons

For a time-invariant stage cost g and $N \to \infty$ the lecture distinguishes:

1. **Total cost:** $J_\pi(x_0) = \lim_{N\to\infty} \mathbb{E}\big[\sum_{k=0}^{N-1} g(x_k, \mu_k(x_k), w_k)\big]$. All stages count equally. This is finite only if the system settles at a zero-cost equilibrium (LQR, Chapter 3).
2. **Discounted cost:** $g_k = \alpha^k g$ with $\alpha \in (0, 1)$. Later costs count less. If $|g| \le M$, then by §1.6
$$|J_\pi(x_0)| \le \sum_{k=0}^{\infty} \alpha^k M = \frac{M}{1-\alpha} < \infty,$$
so the cost is always finite.
3. **Average cost per stage:** $J_\pi(x_0) = \limsup_{N\to\infty} \frac{1}{N} \mathbb{E}\big[\sum_{k=0}^{N-1} g(\cdot)\big]$, the long-run average.

Exercise 2 is about finite N; Task 3 uses discounting with finite N.

---

4. Policies and the optimal cost

4.1 Policies

A **policy** is a sequence of feedback functions $\pi = \{\mu_0, \dots, \mu_{N-1}\}$, $\mu_k : \mathbb{R}^n \to \mathbb{R}^m$. It is **admissible** ($\pi \in \Pi$) if every rule respects the constraints: $\mu_k(x_k) \in U_k(x_k)$ for all x_k and all k.

4.2 Cost of a policy

For a given x_0 and $\pi \in \Pi$, the states are generated by $x_{k+1} = f_k(x_k, \mu_k(x_k), w_k)$ and

$$J_\pi(x_0) := \mathbb{E}\Big[g_N(x_N) + \sum_{k=0}^{N-1} g_k(x_k, \mu_k(x_k), w_k)\Big].$$

4.3 Optimal cost and optimal policy

$$J^\star(x_0) := \min_{\pi \in \Pi} J_\pi(x_0).$$

$J^\star$ is called the **optimal cost function** or **value function**. A policy $\pi^\star$ is **optimal** if $J_{\pi^\star}(x_0) = J^\star(x_0)$ for **every** x_0. This is a strong requirement: one policy must be best from every starting point simultaneously. DP shows that such a policy exists (under the assumptions) and constructs it.

Minimising over Π means minimising over sequences of *functions*, an enormous search space. The rest of the lecture is about avoiding this direct search.

---

5. Bellman's principle of optimality

5.1 Statement

> Any portion of an optimal path is itself optimal (Bellman, 1950s).

Example from the lecture: if the shortest road from Paderborn to Dresden passes through Kassel and Leipzig, then the shortest road from Kassel to Dresden passes through Leipzig, and the shortest road from Paderborn to Leipzig passes through Kassel.

Definition (tail subproblem). Let $\pi^\star = \{\mu_0^\star, \dots, \mu_{N-1}^\star\}$ be optimal. For a stage i and a state x_i, the **tail subproblem** is to minimise the cost from i to N:

$$J_{i,\pi_i}(x_i) := \mathbb{E}\Big[g_N(x_N) + \sum_{k=i}^{N-1} g_k(x_k, \mu_k(x_k), w_k)\Big].$$

Principle: the truncated tail policy $\pi_i^\star = \{\mu_i^\star, \dots, \mu_{N-1}^\star\}$ is optimal for the tail subproblem, from any x_i that can occur with positive probability.

5.2 Why it is true (proof by contradiction, deterministic case)

The total cost of a path splits into a head part (stages 0 to $i-1$) and a tail part (stages i to N): total $=$ head$(x_0 \to x_i)$ + tail$(x_i \to x_N)$.

Suppose the tail of the optimal path were *not* optimal, so another tail from x_i costs strictly less. Keep the head and replace the tail by the cheaper one. The new path starts at x_0, is feasible, and costs strictly less than the "optimal" one. That is impossible, so the tail must be optimal. The head does not affect which tail is cheapest, because the head cost is a constant once x_i is fixed.

5.3 Consequence: history does not matter

In the heating example, suppose the thermometer reads x_i at stage i. It does not matter whether the system got there by starting cold and heating strongly, starting hot and cooling down, or being hit by a draft. The best remaining schedule depends only on the current temperature x_i and the number of steps left $N - i$.

Algorithmic consequence: work backwards.

1. Solve the one-step tail problem at $k = N - 1$ for **every** possible x_{N-1}: $J_{N-1}(x_{N-1}) = \min_{u_{N-1}} \{u_{N-1}^2 + r(x_N - T)^2\}$.
2. Step back to $k = N - 2$: balance u_{N-2}^2 against the already-known $J_{N-1}(x_{N-1})$.
3. Continue down to $k = 0$.

---

6. The DP algorithm

6.1 The algorithm

1. Initialisation ($k = N$):

$$J_N(x_N) = g_N(x_N) \quad \text{for all } x_N.$$

2. Backward recursion ($k = N - 1, N - 2, \dots, 0$):

$$\boxed{J_k(x_k) = \min_{u_k \in U_k(x_k)} \mathbb{E}_{W_k}\Big[g_k(x_k, u_k, W_k) + J_{k+1}(f_k(x_k, u_k, W_k))\Big]}$$

3. Output:

- optimal cost $J^\star(x_0) = J_0(x_0)$;
- optimal policy $\pi^\star = \{\mu_0^\star, \dots, \mu_{N-1}^\star\}$ with

$$\mu_k^\star(x_k) \in \arg\min_{u_k \in U_k(x_k)} \mathbb{E}_{W_k}\Big[g_k(x_k, u_k, W_k) + J_{k+1}(f_k(x_k, u_k, W_k))\Big].$$

How to read the recursion. "The best cost from x_k with $N - k$ stages left = choose u_k to minimise (what this stage costs) + (the best cost from wherever we land), averaged over the disturbance." J_k is called the **cost-to-go**: the cost still to be paid from stage k onward, if we act optimally.

Why this is a breakthrough. The original problem minimises over whole policies $(\mu_0, \dots, \mu_{N-1})$ and integrates over the joint distribution of $(w_0, \dots, w_{N-1})$. Each DP step minimises over a **single** vector u_k and averages over a **single** random variable W_k.

The policy is closed loop. $\mu_k^\star$ is computed for every state x_k, before the process starts (offline). During operation (online) the controller only evaluates $u_k = \mu_k^\star(x_k)$ for the measured state.

6.2 Theorem (optimality of DP) and its proof

> **Theorem.** For every x_0, the optimal cost equals the result of the DP algorithm: $J^\star(x_0) = J_0(x_0)$. Any policy whose $\mu_k^\star(x_k)$ attains the minimum in the recursion for every x_k and k is optimal.

Proof by backward induction. For a tail policy $\pi_k = \{\mu_k, \dots, \mu_{N-1}\}$ write $J_k^\star(x_k)$ for the true optimal cost of the $(N-k)$-stage tail problem starting at x_k:

$$J_k^\star(x_k) = \min_{\pi_k} \mathbb{E}_{W_k,\dots,W_{N-1}}\Big[g_k(x_k, \mu_k(x_k), w_k) + \sum_{i=k+1}^{N-1} g_i(x_i, \mu_i(x_i), w_i) + g_N(x_N)\Big].$$

We show $J_k = J_k^\star$ for all k; for $k = 0$ this is the theorem.

Base case $k = N$. No decisions remain: $J_N^\star(x_N) = g_N(x_N) = J_N(x_N)$.

Induction hypothesis. $J_{k+1}(x) = J_{k+1}^\star(x)$ for all x.

Induction step. Write the tail policy as $\pi_k = (\mu_k, \pi_{k+1})$ and call the bracket above "stage k cost + rest". Then:

1. **Law of total expectation (§1.4):** average first over $W_{k+1}, \dots, W_{N-1}$ with W_k fixed, then over W_k:
$$J_k^\star(x_k) = \min_{(\mu_k, \pi_{k+1})} \mathbb{E}_{W_k}\Big[\mathbb{E}_{W_{k+1},\dots}\big[g_k + \text{rest} \mid W_k\big]\Big].$$
2. **The stage cost is fixed once W_k is fixed** (it depends only on x_k, u_k, w_k), so it leaves the inner expectation:
$$= \min_{(\mu_k, \pi_{k+1})} \mathbb{E}_{W_k}\Big[g_k(x_k, \mu_k(x_k), W_k) + \mathbb{E}_{W_{k+1},\dots}\big[\text{rest} \mid W_k\big]\Big].$$
3. **Independence (§3.3):** given $x_{k+1} = f_k(x_k, \mu_k(x_k), W_k)$, the future disturbances do not depend on W_k. So the inner term is just "the expected tail cost of π_{k+1} starting from x_{k+1}".
4. **Split the minimisation (principle of optimality).** The inner term depends on π_{k+1}, the outer one only on μ_k, and we may minimise the inner term separately for each possible x_{k+1}:
$$= \min_{\mu_k} \mathbb{E}_{W_k}\Big[g_k(x_k, \mu_k(x_k), W_k) + \min_{\pi_{k+1}} \mathbb{E}\big[\text{rest} \mid x_{k+1}\big]\Big].$$
Why "min over everything" equals "min outside, min inside": for any π_{k+1} the inner expected tail cost is $\geq$ its minimum, so the left side is $\geq$ the right side. Conversely, the best tail policy $\pi_{k+1}^\star$ is a set of **feedback functions**, so it achieves the inner minimum for every x_{k+1} at the same time, whatever W_k turns out to be; using it shows the left side is $\leq$ the right side. (This is exactly where feedback matters: an open-loop tail would have to be fixed before W_k is known.)
5. **Definition of $J_{k+1}^\star$:** the inner minimum is $J_{k+1}^\star(x_{k+1})$.
6. **Induction hypothesis:** replace $J_{k+1}^\star$ by J_{k+1}:
$$= \min_{\mu_k} \mathbb{E}_{W_k}\Big[g_k(x_k, \mu_k(x_k), W_k) + J_{k+1}(f_k(x_k, \mu_k(x_k), W_k))\Big].$$
7. **From functions to numbers:** x_k is fixed here, so choosing the function μ_k only matters through the single value $u_k = \mu_k(x_k) \in U_k(x_k)$:
$$= \min_{u_k \in U_k(x_k)} \mathbb{E}_{W_k}\Big[g_k(x_k, u_k, W_k) + J_{k+1}(f_k(x_k, u_k, W_k))\Big] = J_k(x_k).$$

By induction, $J_k = J_k^\star$ for $k = N, N-1, \dots, 0$. The policy built from the minimisers achieves every one of these minima, so its cost is $J_0(x_0) = J^\star(x_0)$: it is optimal. ■

6.3 Complexity and the curse of dimensionality

Discretise the state space $X = [-1,1]^n$ and the input space $U = [-1,1]^m$ with grid width d. Each axis then has $2/d$ points, so

$$|X_d| = \left(\frac{2}{d}\right)^n \text{ states}, \qquad |U_d| = \left(\frac{2}{d}\right)^m \text{ inputs}.$$

- **Brute force (open loop, for one x_0):** try every input sequence of length N: $|U_d|^N = (2/d)^{mN}$ sequences. This grows **exponentially in the horizon** N.
- **DP (for all states at once):** at each of the N stages, for each state, try each input: $N \cdot |X_d| \cdot |U_d| = N\,(2/d)^{n+m}$ evaluations. This grows only **linearly in** N.

Curse of dimensionality. DP is still exponential in the dimensions n and m. With 100 grid points per axis ($d = 0.02$):

- pendulum, $n = 2$: $100^2 = 10^4$ states per stage — easy;
- 6-DOF robot arm, $n = 12$ (6 angles, 6 velocities): $100^{12} = 10^{24}$ states per stage — impossible to store.

Remedies: analytical solutions where the cost-to-go has a known form (linear-quadratic → LQR, Chapter 3 and §10 here), online re-planning (MPC), or approximating J_k by a function (approximate DP / reinforcement learning).

---

7. Worked lecture example: thermal pre-heating

Setup: $x_{k+1} = (1-a)x_k + au_k$, $a \in (0,1)$, cost $r(x_N - T)^2 + \sum_{k=0}^{N-1} u_k^2$, $r > 0$, no disturbance, $u_k \in \mathbb{R}$.

7.1 Initialisation and the last stage $k = N-1$

$k = N$. No inputs remain: $J_N(x_N) = r(x_N - T)^2 =: p_N(x_N - T)^2$ with $p_N := r$.

$k = N-1$. Insert $x_N = (1-a)x_{N-1} + au_{N-1}$:

$$J_{N-1}(x_{N-1}) = \min_{u_{N-1}} \Big\{ u_{N-1}^2 + p_N\big(\underbrace{(1-a)x_{N-1} - T}_{c} + \underbrace{a}_{\beta}\, u_{N-1}\big)^2 \Big\}.$$

$(1-a)x_{N-1}$ is the temperature we would reach with no heating (the **unforced** part); au_{N-1} is what the heater adds (the **forced** part).

Derivative test (same as the key lemma, §1.3, with $r = 1$):

$$\frac{\partial}{\partial u_{N-1}}[\,\dots] = 2u_{N-1} + 2ap_N((1-a)x_{N-1} - T + au_{N-1}) = 0$$

$$\iff \quad (1 + a^2 p_N)\, u_{N-1} = ap_N(T - (1-a)x_{N-1}).$$

The second derivative $2(1 + a^2 p_N) > 0$ confirms a minimum. Hence

$$\mu^\star_{N-1}(x_{N-1}) = \frac{ap_N}{1 + a^2 p_N}(T - (1-a)x_{N-1}),$$

$$J_{N-1}(x_{N-1}) = p_{N-1}\big((1-a)x_{N-1} - T\big)^2, \qquad p_{N-1} = \frac{p_N}{1 + a^2 p_N}.$$

The heater power is proportional to the **predicted shortfall** $T-(1-a)x_{N-1}$: how far below the target we would end without heating.

7.2 Why "unforced" and "forced" terms keep appearing

Step to $k=N-2$ by inserting $x_{N-1}=(1-a)x_{N-2}+au_{N-2}$ into the bracket of J_{N-1}:

$$(1-a)x_{N-1}-T=(1-a)\big((1-a)x_{N-2}+au_{N-2}\big)-T=\underbrace{(1-a)^2x_{N-2}}_{\text{unforced}}+\underbrace{a(1-a)u_{N-2}}_{\text{forced}}-T.$$

In general, with $N-k$ steps left define

- the **unforced terminal temperature** $\widehat{x}_{N|k}:=(1-a)^{N-k}x_k$ (natural cooling of x_k until stage N);
- the **thermal reach** $\widetilde{b}_k:=a(1-a)^{N-1-k}$ (the fraction of heat injected at stage k that survives until N: a enters, then it decays $N-1-k$ times).

7.3 General solution by induction

Claim. For every k: $J_k(x_k)=p_k\big((1-a)^{N-k}x_k-T\big)^2$ with $p_N=r$ and

$$\boxed{p_k=\frac{p_{k+1}}{1+\widetilde{b}_k^2\,p_{k+1}},\qquad \mu_k^\star(x_k)=\widetilde{b}_k\,p_k(T-(1-a)^{N-k}x_k)}$$

Base case $k=N$: $(1-a)^0x_N-T=x_N-T$, and $p_N=r$. ✓

Induction step $k+1\to k$: assume $J_{k+1}(x)=p_{k+1}\big((1-a)^{N-k-1}x-T\big)^2$. Insert $x_{k+1}=(1-a)x_k+au_k$:

$$(1-a)^{N-k-1}\big((1-a)x_k+au_k\big)-T=\underbrace{(1-a)^{N-k}x_k-T}_{c}+\underbrace{a(1-a)^{N-1-k}}_{\widetilde{b}_k}u_k.$$

So $J_k(x_k)=\min_{u_k}\{u_k^2+p_{k+1}(c+\widetilde{b}_ku_k)^2\}$. By the key lemma (§1.3, $r=1$, $\beta=\widetilde{b}_k$, $p=p_{k+1}$):

$$J_k(x_k)=\frac{p_{k+1}}{1+\widetilde{b}_k^2p_{k+1}}\,c^2=p_k\,c^2,\qquad u_k^\star=-\frac{\widetilde{b}_kp_{k+1}}{1+\widetilde{b}_k^2p_{k+1}}\,c=\widetilde{b}_k\,p_k\,(T-(1-a)^{N-k}x_k).\ \checkmark$$

7.4 Physical insight: the inputs ramp up exponentially

Let $e_k:=T-(1-a)^{N-k}x_k$ be the predicted shortfall at stage k, so $u_k^\star=\widetilde{b}_kp_ke_k$.

How does the shortfall evolve? Multiply $x_{k+1}=(1-a)x_k+au_k$ by $(1-a)^{N-k-1}$: $(1-a)^{N-k-1}x_{k+1}=(1-a)^{N-k}x_k+\widetilde{b}_ku_k$. Hence

$$e_{k+1}=e_k-\widetilde{b}_ku_k^\star=e_k(1-\widetilde{b}_k^2p_k).$$

Using $p_k=p_{k+1}/(1+\widetilde{b}_k^2p_{k+1})$:

$$1-\widetilde{b}_k^2p_k=\frac{1+\widetilde{b}_k^2p_{k+1}-\widetilde{b}_k^2p_{k+1}}{1+\widetilde{b}_k^2p_{k+1}}=\frac{1}{1+\widetilde{b}_k^2p_{k+1}}=\frac{p_k}{p_{k+1}}.$$

So $e_{k+1} = e_k\, p_k / p_{k+1}$, and therefore

$$u^\star_{k+1} = \tilde{b}_{k+1}\, p_{k+1}\, e_{k+1} = \tilde{b}_{k+1}\, p_k\, e_k = \frac{\tilde{b}_{k+1}}{\tilde{b}_k}\, u^\star_k = \boxed{\frac{1}{1-a}\, u^\star_k}$$

because $\tilde{b}_{k+1}/\tilde{b}_k = (1-a)^{N-2-k}/(1-a)^{N-1-k} = 1/(1-a)$.

The optimal heating grows by the factor $1/(1-a)$ every stage. This is the balance asked for in §2.3: heat given early leaks away, so less of it is given early; heat given late is efficient but costs quadratically, so it is not all left to the end.

7.5 Numbers ($N = 3$, $a = 0.5$, $r = 16$, $x_0 = 0$, $T = 10$)

Offline backward sweep.

- Thermal reach: $\tilde{b}_2 = 0.5$, $\tilde{b}_1 = 0.5 \cdot 0.5 = 0.25$, $\tilde{b}_0 = 0.5 \cdot 0.5^2 = 0.125$.
- $k = 3$: $p_3 = r = 16$.
- $k = 2$: $p_2 = \dfrac{16}{1 + 0.5^2 \cdot 16} = \dfrac{16}{5} = 3.2$.
- $k = 1$: $p_1 = \dfrac{3.2}{1 + 0.25^2 \cdot 3.2} = \dfrac{3.2}{1.2} = \dfrac{8}{3} \approx 2.667$.
- $k = 0$: $p_0 = \dfrac{8/3}{1 + 0.125^2 \cdot 8/3} = \dfrac{8/3}{1 + 1/24} = \dfrac{8}{3} \cdot \dfrac{24}{25} = \dfrac{64}{25} = 2.56$.

Online forward execution.

- $k = 0$: $u^\star_0 = \tilde{b}_0 p_0 (T - 0.5^3 \cdot 0) = 0.125 \cdot 2.56 \cdot 10 = 3.2$, so $x_1 = 0.5 \cdot 0 + 0.5 \cdot 3.2 = 1.6$.
- $k = 1$: $u^\star_1 = 0.25 \cdot \frac{8}{3} \cdot (10 - 0.25 \cdot 1.6) = \frac{2}{3} \cdot 9.6 = 6.4$, so $x_2 = 0.8 + 3.2 = 4.0$.
- $k = 2$: $u^\star_2 = 0.5 \cdot 3.2 \cdot (10 - 0.5 \cdot 4) = 1.6 \cdot 8 = 12.8$, so $x_3 = 2 + 6.4 = 8.4$.

The inputs double ($1/(1-a) = 2$), as §7.4 predicts. The final temperature 8.4 misses $T = 10$ on purpose: closing the last 1.6 degrees would cost more energy than the terminal penalty saves. Total cost $J_0(0) = p_0(0 - 10)^2 = 256 = 16 \cdot 1.6^2 + 3.2^2 + 6.4^2 + 12.8^2$ ✓ (`python/lecture_examples.py`).

---

8. Further lecture examples

8.1 Deterministic scheduling

Four operations A, B, C, D must be scheduled with A before B and C before D. Starting costs $S_A = 5$, $S_C = 3$; changing from operation m to n costs C_{mn} (values from the lecture slide: $C_{AB} = 3$, $C_{AC} = 2$, $C_{AD} = 6$, $C_{BC} = 4$, $C_{BD} = 2$, $C_{CA} = 1$, $C_{CB} = 5$, $C_{CD} = 4$, $C_{DA} = 2$, $C_{DB} = 3$).

- **State:** the set (in order) of operations completed so far. **Control:** which operation to do next.
- After three operations only one choice remains, so J_3 is the cost of the last transition: $J_3(ABC) = C_{CD} = 4$, $J_3(ACB) = C_{BD} = 2$, $J_3(ACD) = C_{DB} = 3$, $J_3(CAB) = C_{BD} = 2$, $J_3(CAD) = C_{DB} = 3$, $J_3(CDA) = C_{AB} = 3$.
- $k = 2$: $J_2(AB) = 4 + 4 = 8$; $J_2(AC) = \min(5 + 2,\ 4 + 3) = 7$; $J_2(CA) = \min(3 + 2,\ 6 + 3) = 5$; $J_2(CD) = 2 + 3 = 5$.
- $k = 1$: $J_1(A) = \min(3 + 8,\ 2 + 7) = 9$; $J_1(C) = \min(1 + 5,\ 4 + 5) = 6$.

- $k = 0$: $J_0^\star = \min(S_A + J_1(A),\ S_C + J_1(C)) = \min(5 + 9,\ 3 + 6) = 9$.

Following the minimisers forward: $C \to A \to B \to D$ with cost 9. Note how DP **prunes**: at each state only the best continuation is kept.

8.2 Stochastic decision tree: a two-game chess match

Two games; net score x_k (wins minus losses). In each game choose

- **timid**: draw with probability p_d, lose with $1 - p_d$;
- **bold**: win with probability p_w, lose with $1 - p_w$; with $p_d > p_w$.

Goal: maximise the probability of winning the match. (To make the end-game concrete, assume as in Bertsekas' version of this example that a tie after two games goes to a sudden-death game, which is played bold and won with probability p_w.) So $J_2(x_2) = 1$ if $x_2 > 0$, p_w if $x_2 = 0$, 0 if $x_2 < 0$, and the DP uses max instead of min.

Game 2 ($k = 1$):

- $x_1 = +1$: timid gives $p_d \cdot 1 + (1 - p_d)\, p_w$; bold gives $p_w \cdot 1 + (1 - p_w)\, p_w$. Timid minus bold $= (p_d - p_w)(1 - p_w) > 0 \Rightarrow$ **timid** (protect the lead).
- $x_1 = 0$: timid gives $p_d\, p_w$; bold gives p_w. Bold is better since $p_d < 1 \Rightarrow$ **bold**.
- $x_1 = -1$: timid gives 0 (a draw still loses); bold gives $p_w^2 \Rightarrow$ **bold**.

Game 1 ($k = 0$, $x_0 = 0$): timid gives $p_d J_1(0) + (1 - p_d) J_1(-1) = p_d p_w + (1 - p_d) p_w^2$; bold gives $p_w J_1(1) + (1 - p_w) J_1(-1) = p_w(p_d + (1 - p_d) p_w) + (1 - p_w) p_w^2$. Bold minus timid $= p_w^2 (1 - p_w) > 0 \Rightarrow$ **play bold**, then switch to timid if you won.

With $p_w = 0.45$, $p_d = 0.9$ the match is won with probability $0.537 > 0.5$, although each bold game is lost more often than won. The flexibility to adapt game 2 to the result of game 1 creates this advantage — the value of feedback again.

8.3 Stochastic inventory control

Stock x_k, order $u_k \geq 0$, random demand $w_k \geq 0$: $x_{k+1} = x_k + u_k - w_k$. Stage cost $c u_k + r(x_k + u_k - w_k)$ with r a convex holding/shortage penalty.

DP: $J_k(x) = \min_{u \geq 0} \mathbb{E}\big[cu + r(x + u - w) + J_{k+1}(x + u - w)\big]$. Substitute the **order-up-to level** $y = x + u$ (stock after ordering), which must satisfy $y \geq x$:

$$J_k(x) = -cx + \min_{y \geq x} G_k(y), \qquad G_k(y) := cy + \mathbb{E}\big[r(y - w) + J_{k+1}(y - w)\big].$$

If G_k is convex with unconstrained minimiser $S_k^\star$: when $x < S_k^\star$, the best feasible y is $S_k^\star$; when $x \geq S_k^\star$, G_k increases for $y \geq x$, so $y = x$ (order nothing). This is the **base-stock policy**

$$\mu_k^\star(x_k) = \max\big(0,\ S_k^\star - x_k\big).$$

One can show that J_k is again convex, so the argument repeats at every stage. The point for us: DP can reveal the **structure** of the optimal policy (here a threshold rule) without discretising anything.

---

9. Designing cost functions

The cost is how an engineer tells the optimiser what "good" means. Each term should encode one objective or constraint.

9.1 Quadratic tracking cost and its weights

$g_k(x_k, u_k) = \|x_k - x_{\text{ref}}\|^2 + r\|u_k\|^2$, $g_N(x_N) = \|x_N - x_{\text{ref}}\|^2$ ($\|v\|^2 = v^\top v$):

- the first term penalises distance from the reference (accuracy);
- the second term penalises effort (energy, actuator wear, saturation risk);
- squares punish large deviations much more than small ones and give smooth, differentiable problems;
- the ratio of the weights sets the trade-off: small r ⇒ aggressive control, large r ⇒ gentle control.

Timing: u_k cannot change x_k (it acts after x_k is measured). It affects $x_{k+1}, x_{k+2}, \ldots$. So u_k is chosen by trading its own cost $r\|u_k\|^2$ against the **future** state penalties.

9.2 The terminal cost

g_N prices the final state only. A large terminal weight (for example $g_N = q\,x_N^2$ with $q \gg 1$) says "ending close to zero matters more than the path". The script points to Exercise 2 for this. With $q_N = 0$ the final state is not priced at all, and the last decision has no reason to act on the state (Task 2(b), numerical experiment).

9.3 Hard constraints

Two equivalent ways to forbid states or inputs:

- restrict the admissible set, $u_k \in U_k(x_k)$ (only inputs that keep the next state feasible);
- give forbidden points cost $+\infty$ (**barrier / indicator penalty**): $g(x, u) = x^2 + u^2$ if $|x| \le \bar{x}$, else $+\infty$. Any policy that can reach $|x| > \bar{x}$ has infinite expected cost, so DP avoids it at all costs.

The danger: if disturbances can push the state over the limit no matter what the controller does, **every** policy has infinite cost and the problem has no meaningful solution.

9.4 Soft constraints

$g(x, u) = x^2 + u^2 + \lambda \max(0, |x| - \bar{x})^2$, $\lambda > 0$:

- inside $[-\bar{x}, \bar{x}]$ the extra term is zero (a **dead zone**);
- outside, the cost grows smoothly with the size of the violation, and λ sets how strongly.

Violations are allowed but discouraged, and the cost stays finite, so small disturbance-driven violations do not break the problem.

9.5 Economic objectives

$g(x, u) = -\big(\text{Revenue}(x) - \text{Cost}(x, u)\big)$: minimising the negative profit is maximising profit (§1.1). Stage costs are then often negative, which is allowed (§3.2).

9.6 State augmentation

DP requires the stage cost and dynamics to depend only on (x_k, u_k, w_k). If something else from the past is needed (a previous input, a delayed state, a forecast, a correlated disturbance), **add it to the state**. The general recipe:

1. Identify the extra memory z_k that the cost or dynamics need at stage k.
2. Define $\tilde{x}_k = (x_k, z_k)$.
3. Write the dynamics of z so that z_{k+1} is a function of (x_k, z_k, u_k, w_k).

4. Express the cost as a function of $(\tilde{x}_k, u_k, w_k)$.

Example (time delay): if $x_{k+1} = f(x_k, u_{k-1}, w_k)$, use $\tilde{x}_k = (x_k, u_{k-1})$, with $\tilde{x}_{k+1} = (f(x_k, u_{k-1}, w_k),\ u_k)$. The price: the state dimension grows, and with it the curse of dimensionality (§6.3).

---

10. Linear-quadratic DP (scalar)

10.1 Deterministic case

System $x_{k+1} = ax_k + bu_k$ ($b \neq 0$), cost

$$q_N\, x_N^2 + \sum_{k=0}^{N-1} (q\, x_k^2 + r\, u_k^2), \qquad q \geq 0,\ r > 0,\ q_N \geq 0.$$

Claim: $J_k(x) = p_k x^2$ for every k, with $p_N = q_N$ and

$$\boxed{p_k = q + \frac{r\, a^2\, p_{k+1}}{r + b^2 p_{k+1}}, \qquad \mu_k^\star(x) = -L_k x, \quad L_k = \frac{a\, b\, p_{k+1}}{r + b^2 p_{k+1}}}$$

Base case: $J_N(x) = q_N x^2 = p_N x^2$. ✓

Induction step: assume $J_{k+1}(x) = p_{k+1} x^2$. Then

$$J_k(x) = \min_u \{qx^2 + ru^2 + p_{k+1}(ax + bu)^2\} = qx^2 + \min_u \{ru^2 + p_{k+1}(\underbrace{a}_{c}\underbrace{x}_{} + \underbrace{b}_{\beta} u)^2\}.$$

The key lemma (§1.3) gives

$$J_k(x) = qx^2 + \frac{r\, p_{k+1}}{r + b^2 p_{k+1}}\, a^2 x^2 = p_k x^2, \qquad u^\star = -\frac{b\, p_{k+1}}{r + b^2 p_{k+1}}\, ax. \checkmark$$

Notes:

- $p_k \geq q \geq 0$ for $k < N$, so the second derivative $2(r + b^2 p_{k+1})$ is always positive: each step is a strictly convex problem.
- **Riccati form.** Since $\dfrac{ra^2 p}{r + b^2 p} = a^2 p - \dfrac{a^2 b^2 p^2}{r + b^2 p}$ (put both over the denominator $r + b^2 p$), the recursion can also be written $p_k = q + a^2 p_{k+1} - \dfrac{a^2 b^2 p_{k+1}^2}{r + b^2 p_{k+1}}$. This is the scalar version of the Riccati recursion of Chapter 3.
- **Time-varying gains.** L_k changes with k even though the system and stage cost are time-invariant, because the number of remaining steps changes. Near the end the terminal weight dominates; far from the end, p_k settles (numerical experiment).
- **Optimal cost:** $J^\star(x_0) = p_0 x_0^2$.

10.2 What the terminal weight does at the last step

At $k = N-1$, $L_{N-1} = \dfrac{ab\,q_N}{r + b^2 q_N}$ and the final state is

$$x_N = (a - bL_{N-1})\,x_{N-1} = a\Big(1 - \frac{b^2 q_N}{r + b^2 q_N}\Big)x_{N-1} = \frac{a\,r}{r + b^2 q_N}\,x_{N-1}.$$

- $q_N = 0$: $L_{N-1} = 0$, no action at the last step, $x_N = a x_{N-1}$ (the state just drifts).
- $q_N \to \infty$: $L_{N-1} \to a/b$, so $x_N \to 0$. This is **deadbeat** control: the last input cancels the state exactly.

10.3 Additive noise and certainty equivalence

Now $x_{k+1} = a x_k + b u_k + w_k$ with independent w_k, $\mathbb{E}[w_k] = 0$, $\mathbb{E}[w_k^2] = \sigma^2$.

Claim: $J_k(x) = p_k x^2 + c_k$ with the **same** p_k as in §10.1, $c_N = 0$ and $c_k = c_{k+1} + p_{k+1}\sigma^2$.

Induction step: assume $J_{k+1}(x) = p_{k+1}x^2 + c_{k+1}$. With $m := ax + bu$ (fixed when averaging over w_k) and §1.4:

$$\mathbb{E}\big[J_{k+1}(m + w_k)\big] = p_{k+1}\mathbb{E}\big[(m + w_k)^2\big] + c_{k+1} = p_{k+1}m^2 + p_{k+1}\sigma^2 + c_{k+1}.$$

So

$$J_k(x) = \min_u \big\{qx^2 + ru^2 + p_{k+1}(ax + bu)^2\big\} + \underbrace{p_{k+1}\sigma^2 + c_{k+1}}_{\text{does not depend on } u}.$$

The constant leaves the minimisation (§1.1). The remaining problem is exactly the deterministic one, so the minimiser and p_k are unchanged, and $c_k = c_{k+1} + p_{k+1}\sigma^2$. ✓

Certainty equivalence: for linear dynamics, quadratic cost and additive zero-mean noise, the optimal feedback is the same as if the noise were replaced by its mean (zero). The noise only adds the unavoidable cost $c_0 = \sigma^2 \sum_{k=0}^{N-1} p_{k+1}$. Intuition: the controller acts before w_k is known and the noise enters additively, so the best it can do is aim for where the average says the state will be; the spread around that point cannot be influenced by u_k.

10.4 Preview: the matrix case (LQR)

For $x_{k+1} = Ax_k + Bu_k$ and cost $x_N^\top Q_N x_N + \sum_k (x_k^\top Q x_k + u_k^\top R u_k)$ the same induction gives $J_k(x) = x^\top P_k x$ with a matrix Riccati recursion and $u_k^\star = -K_k x_k$. Because the cost-to-go is known to be quadratic, no state grid is needed: no curse of dimensionality. This is Chapter 3.

---

11. Discounted cost

11.1 Finite-horizon discounted problem

$$J_\pi(x_0) = \mathbb{E}\Big[\alpha^N g_N(x_N) + \sum_{k=0}^{N-1} \alpha^k g_k(x_k, u_k, w_k)\Big], \qquad \alpha \in (0, 1).$$

This is a standard DP problem with stage cost $\tilde{g}_k = \alpha^k g_k$ and terminal cost $\tilde{g}_N = \alpha^N g_N$, so §6 applies directly: $J_N = \alpha^N g_N$, $J_k(x) = \min_u \mathbb{E}\big[\alpha^k g_k(x, u, W_k) + J_{k+1}(f_k(x, u, W_k))\big]$.

Why discount? Money now is worth more than money later; the far future is more uncertain; and for $N \to \infty$ discounting keeps the cost finite (§3.5).

11.2 The current-value function

The factor α^k makes J_k shrink with k even when nothing else changes. Rescaling removes it: define $V_k(x) := \alpha^{-k} J_k(x)$ ("cost-to-go measured in stage-k units"). Then

$$V_N = g_N, \qquad V_k(x) = \min_{u \in U_k(x)} \mathbb{E}\Big[g_k(x, u, W_k) + \alpha\, V_{k+1}(f_k(x, u, W_k)) \Big]$$

(derived step by step in Solutions, Task 3). Each stage now contributes its cost undiscounted, and the future is discounted by **one** factor α. Multiplying by the positive constant α^{-k} does not change the argmin, so V_k and J_k give the same optimal policy.

11.3 The Bellman operator

For a time-invariant problem ($f_k = f$, $g_k = g$, $U_k = U$, i.i.d. W_k) define the operator T that maps a function V to a new function:

$$(TV)(x) := \min_{u \in U(x)} \mathbb{E}\big[g(x, u, W) + \alpha V(f(x, u, W)) \big].$$

The current-value recursion is simply $V_k = TV_{k+1}$, so $V_k = T^{N-k} g_N$: the **same** operator applied $N - k$ times. V_k depends only on the number of remaining steps, not on the calendar time k. This is the bridge to the infinite horizon (§14).

---

12. Tabular DP with hard constraints

When states and inputs are finite (or the constraints make derivatives useless), DP is done **by table**:

1. List all states at each stage and fill in $J_N(x) = g_N(x)$.
2. For $k = N - 1$ down to 0, for each state x:
   - list the **admissible** inputs, $u \in U_k(x)$ (the constraint is enforced here by simply not listing infeasible inputs);
   - for each, compute the next state $f_k(x, u)$ and the value $g_k(x, u) + J_{k+1}(f_k(x, u))$ (with an average over w if there is noise);
   - write down the smallest value as $J_k(x)$ and the input(s) achieving it as $\mu_k^\star(x)$.
3. Run forward from x_0, applying $\mu_k^\star$, to read off the optimal schedule.

If two inputs tie, both are optimal; record both. Different tie-breaks give different but equally cheap trajectories. A practical rule (for example "prefer the full battery") can decide.

In the stage-0 step only the actual initial state needs to be evaluated, because x_0 is known.

---

13. Monotonicity of DP

Lemma (monotonicity of the Bellman operator). Let $(T_k J)(x) := \min_{u \in U_k(x)} \mathbb{E}\big[g_k(x, u, W_k) + J(f_k(x, u, W_k)) \big]$. If $J(y) \le J'(y)$ for all y, then $(T_k J)(x) \le (T_k J')(x)$ for all x.

Proof.

1. For fixed x and u, the inequality holds at every possible next state: $J(f_k(x, u, w)) \le J'(f_k(x, u, w))$ for every w.

2. An expectation (a weighted sum with non-negative weights) preserves inequalities: $\mathbb{E}[g + J(f)] \le \mathbb{E}[g + J'(f)]$.
3. A minimum preserves inequalities: let $h(u) \le h'(u)$ for all u. For any u, $\min_v h(v) \le h(u) \le h'(u)$; so $\min_v h(v)$ is a lower bound of h' and hence $\le \min_u h'(u)$. ■

In words: a more pessimistic estimate of the future can never make the present look better. The additional task uses this, together with time invariance ($T_k = T$ for all k), to show by induction that the cost-to-go functions are ordered in k.

---

14. Outlook: infinite horizon

For a fixed deadline (pre-heat by stage N) the backward recursion starts at $J_N = g_N$. For continuous operation (keep a room warm forever) there is no last stage to start from.

Stationarity. With time-invariant f, g and an infinite horizon, looking ahead from stage k is the same as looking ahead from stage $k+1$. The recursion $V_k = TV_{k+1}$ turns into a **fixed-point equation**

$$J^\star = TJ^\star,$$

and both $J^\star$ and the policy $\mu^\star$ become time-invariant.

Contraction (discounted case). Let $\|J\|_\infty = \max_x |J(x)|$. For any J, J', any x and any u: $\mathbb{E}[g + \alpha J(f)] \le \mathbb{E}[g + \alpha J'(f)] + \alpha\|J - J'\|_\infty$. Minimising both sides over u gives $TJ \le TJ' + \alpha\|J - J'\|_\infty$; swapping J and J' gives the reverse, so

$$\|TJ - TJ'\|_\infty \le \alpha \, \|J - J'\|_\infty.$$

T shrinks distances by the factor $\alpha < 1$. Consequently $J^\star = TJ^\star$ has exactly one solution, and **value iteration** $J_{j+1} = TJ_j$ converges to it from any start, with error $\|J_j - J^\star\|_\infty \le \alpha^j \|J_0 - J^\star\|_\infty$ (geometric convergence). `python/task3_discounted_dp.py` shows the successive differences shrinking by roughly α per iteration.

---

15. Cheat sheet

Concept	Formula / rule
System	$x_{k+1} = f_k(x_k, u_k, w_k)$, $u_k \in U_k(x_k)$
Cost	$\mathbb{E}[g_N(x_N) + \sum_{k=0}^{N-1} g_k(x_k, u_k, w_k)]$
Policy	$\pi = \{\mu_0, \dots, \mu_{N-1}\}$, $u_k = \mu_k(x_k)$
Optimal cost	$J^\star(x_0) = \min_{\pi \in \Pi} J_\pi(x_0)$
Principle of optimality	tails of optimal policies are optimal for the tail problems
DP initialisation	$J_N = g_N$
DP recursion	$J_k(x) = \min_{u \in U_k(x)} \mathbb{E}_{W_k}[g_k(x, u, W_k) + J_{k+1}(f_k(x, u, W_k))]$
DP policy	$\mu_k^\star(x) \in$ arg min of the recursion
Complexity	brute force $(2/d)^{mN}$; DP $N(2/d)^{n+m}$
Key lemma	$\min_u[ru^2 + p(c + \beta u)^2] = \frac{rp}{r+\beta^2 p}c^2$ at $u = -\frac{\beta p}{r+\beta^2 p}c$
Zero-mean noise	$\mathbb{E}[p(m + W)^2] = pm^2 + p\sigma^2$
Heating weights	$p_N = r$, $p_k = p_{k+1}/(1 + \tilde{b}_k^2 p_{k+1})$, $\tilde{b}_k = a(1-a)^{N-1-k}$
Heating policy	$\mu_k^\star = \tilde{b}_k p_k (T - (1-a)^{N-k} x_k)$, $u_{k+1}^\star = u_k^\star/(1-a)$

Concept	Formula / rule
Scalar LQ	$p_N = q_N$, $p_k = q + \frac{ra^2 p_{k+1}}{r+b^2 p_{k+1}}$, $u_k = -\frac{abp_{k+1}}{r+b^2 p_{k+1}} x_k$
LQ + noise	$J_k = p_k x^2 + c_k$, $c_k = c_{k+1} + p_{k+1}\sigma^2$, same gains
Last-step state	$x_N = \frac{ar}{r+b^2 q_N} x_{N-1}$; $q_N = 0$: no action; $q_N \to \infty$: deadbeat
Discounted DP	$\tilde{g}_k = \alpha^k g_k$, $V_k = \alpha^{-k} J_k$, $V_k = \min \mathbb{E}[g + \alpha V_{k+1}(f)]$
Bellman operator	$(TV)(x) = \min_u \mathbb{E}[g + \alpha V(f)]$, $V_k = T^{N-k} g_N$
Monotonicity	$J \leq J' \Rightarrow TJ \leq TJ'$
Contraction	$\|TJ - TJ'\|_\infty \leq \alpha \|J - J'\|_\infty$
Barrier penalty	cost $+\infty$ outside the feasible set (hard)
Soft penalty	$\lambda \max(0, \|x\| - \bar{x})^2$ (finite, smooth)
State augmentation	add needed memory to the state: $\tilde{x}_k = (x_k, z_k)$
Base-stock policy	$\mu_k^\star(x) = \max(0, S_k^\star - x)$